Увод в програмирането

Функции в C++

ФМИ, специалност „Софтуерно инженерство“

Съдържание

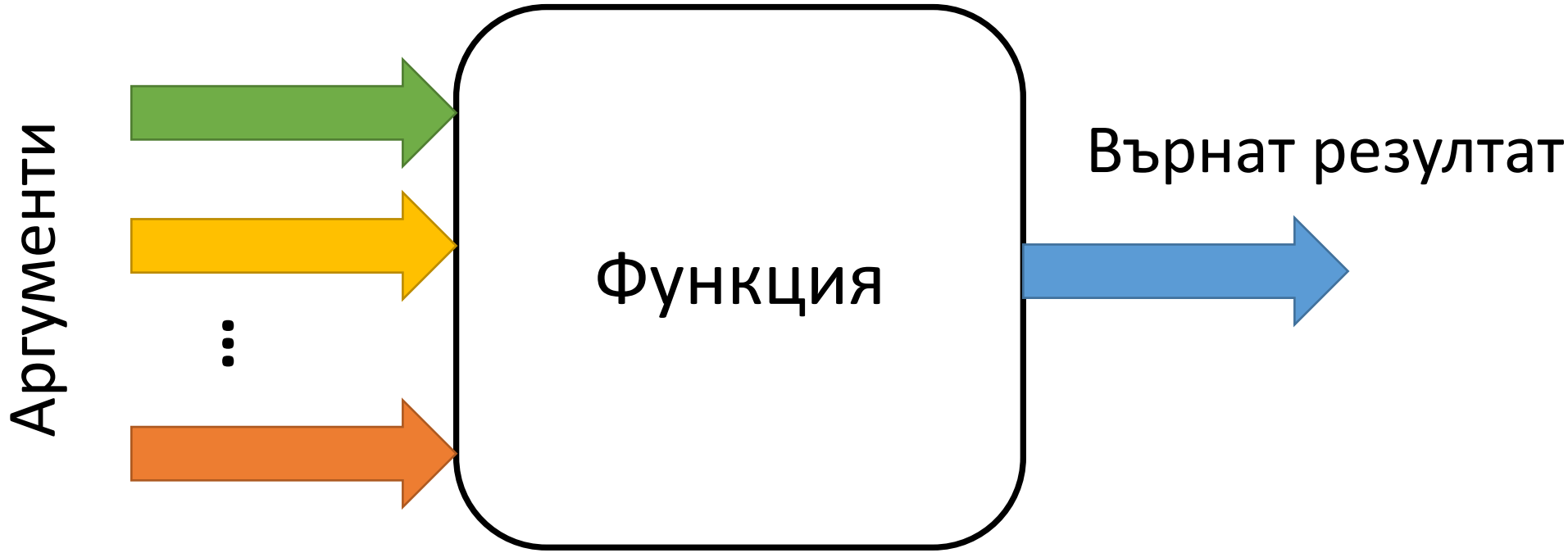
- Понятие за функция в C++
- Деклариране, дефиниране и извикване на функции
- Формални и фактически параметри
- Подаване на параметри
- Предефиниране на функции
- Контрол на входните данни
- Функцията main()

Функции

- Какво означава функция
  - В математиката (най-общо) – съпоставяне на определени величини, наричани *аргументи*, на други величини, наричани *стойности*, като на дадени аргументи се съпоставя точно една стойност
- В програмирането?

Функции

- В езика C++, функциите са основен конструктивен елемент на програмите
- Дават възможност за многократна употреба
  - Всяка функция се състои от множество от оператори, оформени подходящо за да се използват като обобщено действие или операция.
  - След като една функция бъде дефинирана, тя може да бъде изпълнявана многократно за различни входни данни.


Аргументи
...
Функция
Върнат резултат

Функции

- Разделяй и владей
  - Позволяват да се създадават програми от комбиниране на по-малки елементи (т.е. Функции)
  - Всеки такъв елемент е по-ясен и лесен за тестване и модифициране
- Избягва се многократното повтаряне на едни и същи програмни фрагменти. Те се дефинират еднократно като функции, след което могат да бъдат изпълнявани произволен брой пъти.
- В повечето случаи се постига икономия на памет, тъй като кодът на функцията се съхранява само на едно място в паметта, независимо от броя на нейните изпълнения.

Дефиниране на функция в C++

- <сигнатура> { <тяло> }
- <сигнатура> ::= [<тип_резултат> | void]
  <идентификатор> (<формални_параметри>)
  - void = празен тип, не връща резултат
- <формални_параметри> ::= <без параметри> | void | <параметър> {, <параметър> }
- <параметър> ::= <тип> [<идентификатор>]
- <тяло> ::= { <оператор> }

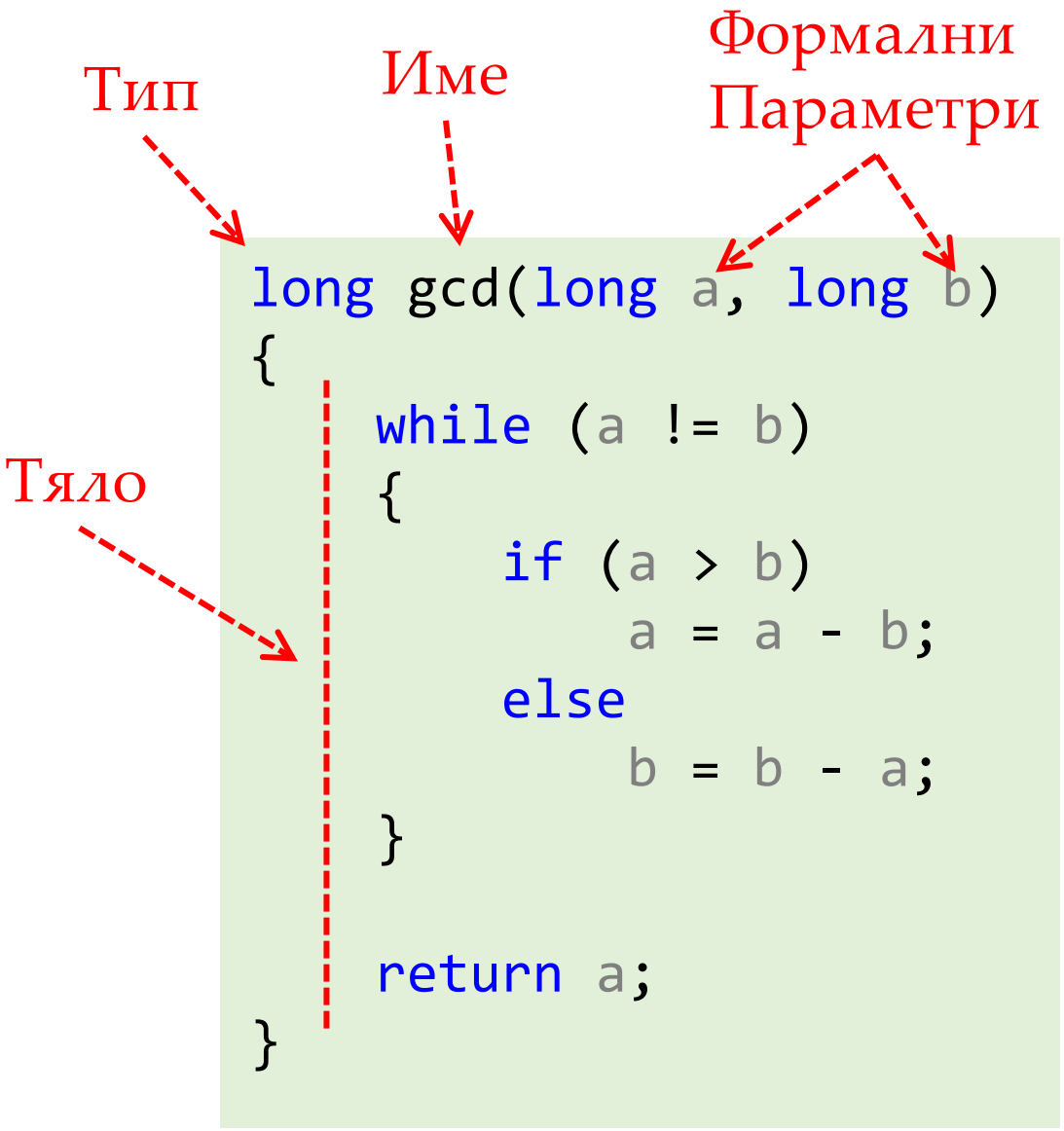
Извикване на функция

- Със стартиране на програмата се извиква главната функция main()
- След това всяка функция, включително и main() може да извиква произволен брой други функции
- Извикването на функция е операция с много висок приоритет
- Извикващата функция продължава изпълнението само след като извиканата функция завърши
  - Всяка функция завършва при срещане на } , която маркира край на тялото ѝ, или при изпълнение на оператора return;

Извикване на функция

- <име>(<фактически_параметри>)
- <фактически_параметри>::=<празно> | void | <израз> {, <израз> }
- <формален_параметър> = <фактически_параметър>
  - ако се налага, прави се преобразуване на типовете
  - типът на фактическия параметър се съпоставя с типа на съответния формален параметър
- Фактическите параметри се наричат още атрибути

Дефиниция на функция



Извикване на функция

Име на функцията,
която извикваме

```
gcd(130, 50);
```

Фактически параметри
(Аргументи)

Примери за функции в C++

- Да се намери абсолютната стойност на дадено цяло число

```
int AbsoluteValue(int number) {
   int retValue;
   retValue = (number >= 0) ? number : -number;
   return retValue;
}
```

- Извикване на функцията

```
int main() {
  int aNumber;
  cout << "Input a number: "; cin >> aNumber;
  aNumber = AbsoluteValue(aNumber);
  cout << "The absolute value is: " << aNumber << endl;
}
```

Алтернативи

```
int AbsoluteValue(int number) {
    return (number >= 0) ? number : -number;
}


int main() {
    int aNumber;
    cout << "Input a number: "; cin >> aNumber;

    cout << "The absolute value is: " << AbsoluteValue(aNumber) << endl;
}
```

```
// Една функция може и да не връща нищо
void PrintLine(unsigned int Size)
{
   for(int i=0; i < Size; i++)
   {
      cout << '-';
   }
}

int main()
{
   PrintLine(10); // OK
   int x = PrintLine(20); // Грешка!
}
```

```
int MyFunction(int a)
{
    // Ако функцията има тип (в случая – int),то
    // всички пътища на изпълнение трябва да връщат
    // стойност от този тип!
    if (a > 0)
    {
        // a is positive
        return a;
    }
    else
    {
        // a is negative
    }
}
```

```
long gcd(long a, long b)
{
    while (a != b)
    {
        if (a > b)
            a = a - b;
        else
            b = b - a;
    }

    return a;
}
```

```
long gcd(long a, long b)
{
    if (a == 0 || b == 0)
        return AbsoluteValue(a-b);
    if ( a < 0 ) a = AbsoluteValue(a);
    if ( b < 0 ) b = AbsoluteValue(b);

    while (a != b)
    {
        if (a > b)
            a = a - b;
        else
            b = b - a;
    }

    return a;
}
```

```
// Предполагаме, че ReadPositiveIntFromFile()
// е функция, която прочита положително число от
// някакъв файл и връща -1 ако не успее.

void MyFunction()
{
    int Data = ReadPositiveIntFromFile();

    if (Data == -1)
    {
        cout << "ERROR: Cannot read from file!\n";
        return;
    }

    // Тук използваме прочетените стойности ...
}
```

Дефиниция и декларация на функция

- Декларацията на функцията представлява нейната сигнатура, последвана от ;
  - Не е задължителна
  - Може да се каже, че декларацията е своеобразно „обещание“ за дефиниране на функцията по-късно в кода на програмата
- Дефиницията на функцията описва и нейното тяло
- Декларацията на дадена функция задължително трябва да предшества извикването и в друга функция
- Защо се налага да правим разлика между декларация и дефиниция

```
int f();
int g(int, int); // В декларацията можем да пропуснем
                 // имената на параметрите

int main()
{
    int a = f();
    int b = g(5, 10);
}

int f()
{
    return 10;
}

int g(int a, int b)
{
    return a + b * 100;
}
```

```
int f()                     // Дефиниция на f()
{
    return 10;
}

int main()
{
    int a = f();            // ОК – f() е дефинирана по-горе
    int b = g(5, 10); // Грешка! Не е ясно какво е g?
}

int g(int a, int b)    // Дефиниция на g()
{
    return a + b * 100;
}
```

```
int f()                     // Дефиниция на f()
{
    return 10;
}

int g(int a, int b)    // Дефиниция на g()
{
    return a + b * 100;
}

int main()
{
    int a = f();          // OK – f() е дефинирана по-горе
    int b = g(5, 10); // OK – g() е дефинирана по-горе
}
```

```
int f();                // Декларация на f()
int g(int a, int b);    // Декларация на g()

int main()
{
    int a = f();        // OK – функцията f() е декларирана по-горе
    int b = g(5, 10);   // OK – функцията g() е декларирана по-горе
}

int f()                 // Дефиниция на f()
{
    return 10;
}

int g(int a, int b)     // Дефиниция на g()
{
    return a + b * 100;
}
```

```
int f();                // Декларация на f()
int g(int a, int b);    // Декларация на g()

int main()
{
    int a = f();        // OK – функцията f() е декларирана по-горе
}

int f()                 // Дефиниция на f()
{
    return g(10, 2); // OK – функцията g() е декларирана по-горе
}

int g(int a, int b)     // Дефиниция на g()
{
    return a + b * 100;
}
```

Област на видимост

```
void f()
{
    int b = 1000;
    a = 2000; // Грешка!
}
```

Област на видимост на b

```
int main()
{
    int a = 1;
    f();
    b = 2; // Грешка!
    return 0;
}
```

Област на видимост на a

```cpp
#include <iostream>
using namespace std;

int c;

void f() {
    c = 200;
}

int main() {
    c = 100;
    f();
    cout << c << endl;
    return 0;
}
```

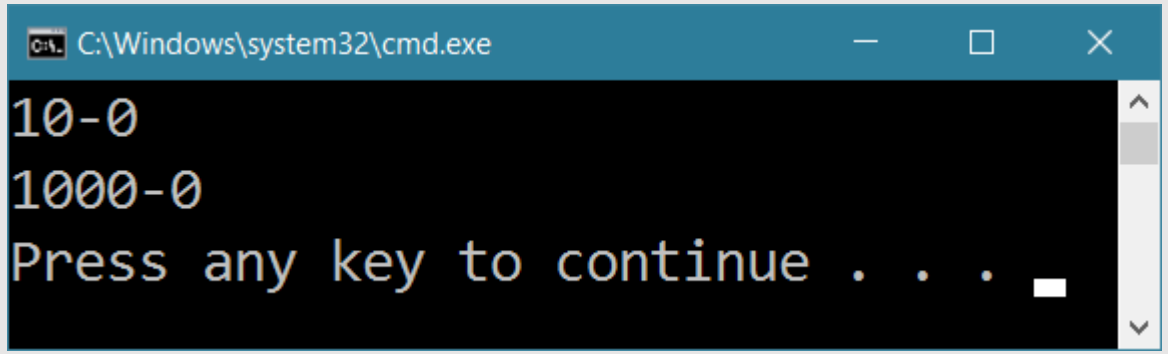
Област на видимост на c

```cpp
#include <iostream>
using namespace std;

int a = 0;

void f()
{
    int a = 10;
    cout << a << "-" << ::a << endl; // Извежда 10-0
}

int main()
{
    int a = 1000;
    f();
    cout << a << "-" << ::a << endl; // Извежда 1000-0
}
```

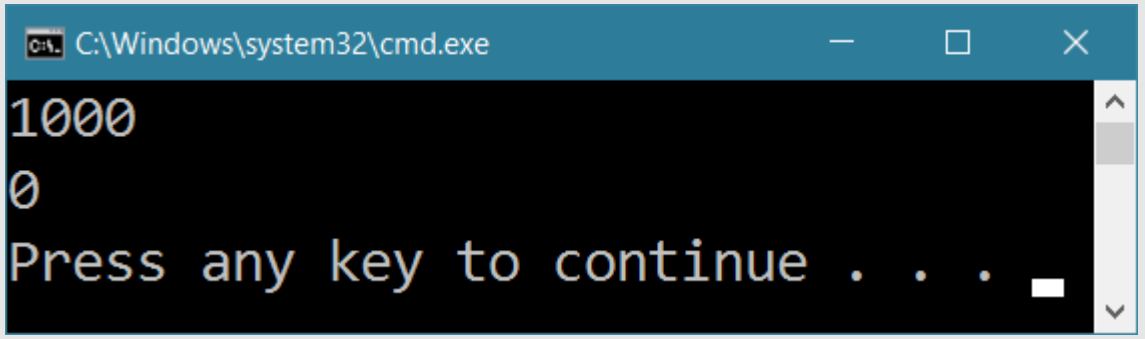


Характерно за параметрите на функциите

- Пресмята се стойността на фактическия параметър
- В специално място в паметта (наречено стекова рамка на функцията) се създава копие на стойността
- След това, всяка промяна на стойността остава локална за функцията
- При завършване на функцията,предадената стойност и всички промени над нея се загубват

```
// Предаване на параметри по стойност
// Функцията работи върху копие на оригиналните данни!
void f(int a)
{
    a = 1000;
    cout << a << endl;
}

int main()
{
    int a = 0;
    f(a);
    cout << a << endl;
    return 0;
}
```



- А ако искаме да променяме стойностите на аргументите?
  - Например да напишем функция, която разменя стойностите на две числа

Тип указател и тип псевдоним

- Тип указател

```
int a = 10;
int *pa;
```

- Тип псевдоним (reference)

```
int a;
int &ra = a;
```

Предаване на аргументи по псевдоним

```
void swap(int& x, int& y) {
  int tmp = x;
  x = y;
  y = tmp;
}

int main() {
  int a = 5, b = 8;
  swap(a, b);
  cout << a << ' ' << b << endl;
}
```

```
// Предефиниране на функция (overload)

void MyFunction(int a)
{
    cout << "MyFunction (int)\n";
}

void MyFunction(double a)
{
    cout << "MyFunction (double)\n";
}

double MyFunction(int a, double b)
{
    cout << "MyFunction (int, double)\n";
    return (double)a * 10.0 + b;
}
```

```
int a = 5;
double b = 10;

MyFunction(a);              // MyFunction(int)
MyFunction(b);              // MyFunction(double)
MyFunction(a, b);           // MyFunction(int, double)

MyFunction(10);             // MyFunction(int)

MyFunction(10.0);           // MyFunction(double)
MyFunction((double)10); // MyFunction(double)

MyFunction(10, 10.0);   // MyFunction(int, double)
MyFunction(10.0, 10);   // MyFunction(double), WARNING!
```

```
// Функция НЕ МОЖЕ да се предефинира само по типа на
// върнатия резултат.
// Трябва да има и разлика в броя или вида на аргументите.

int MyFunction(int a)
{
    cout << "MyFunction(int)\n";
}


double MyFunction(int a)
{
    cout << "MyFunction(double)\n";
}

int main()
{
    MyFunction(10); // Един пример защо това не е позволено.
                    // Коя версия бихме извикали тук?
}
```

Параметри по подразбиране

```
int MyFunction(int a, int b = 0)
{
    return a + b;
}

int main()
{
    int x = MyFunction(10, 20); // x = 30;
    int y = MyFunction(10);     // y = 10;
    int z = MyFunction();       // Грешка!
}
```



```
// Подразбиращите се параметри трябва винаги
// да бъдат в края!

int MyFunction(int a = 0, int b) // Грешка!
{
    return a + b;
}
```

```
int MyFunction(int a = 10, int b = 10)
{
    return a + b;
}

int MyFunction()
{
    return 0;
}

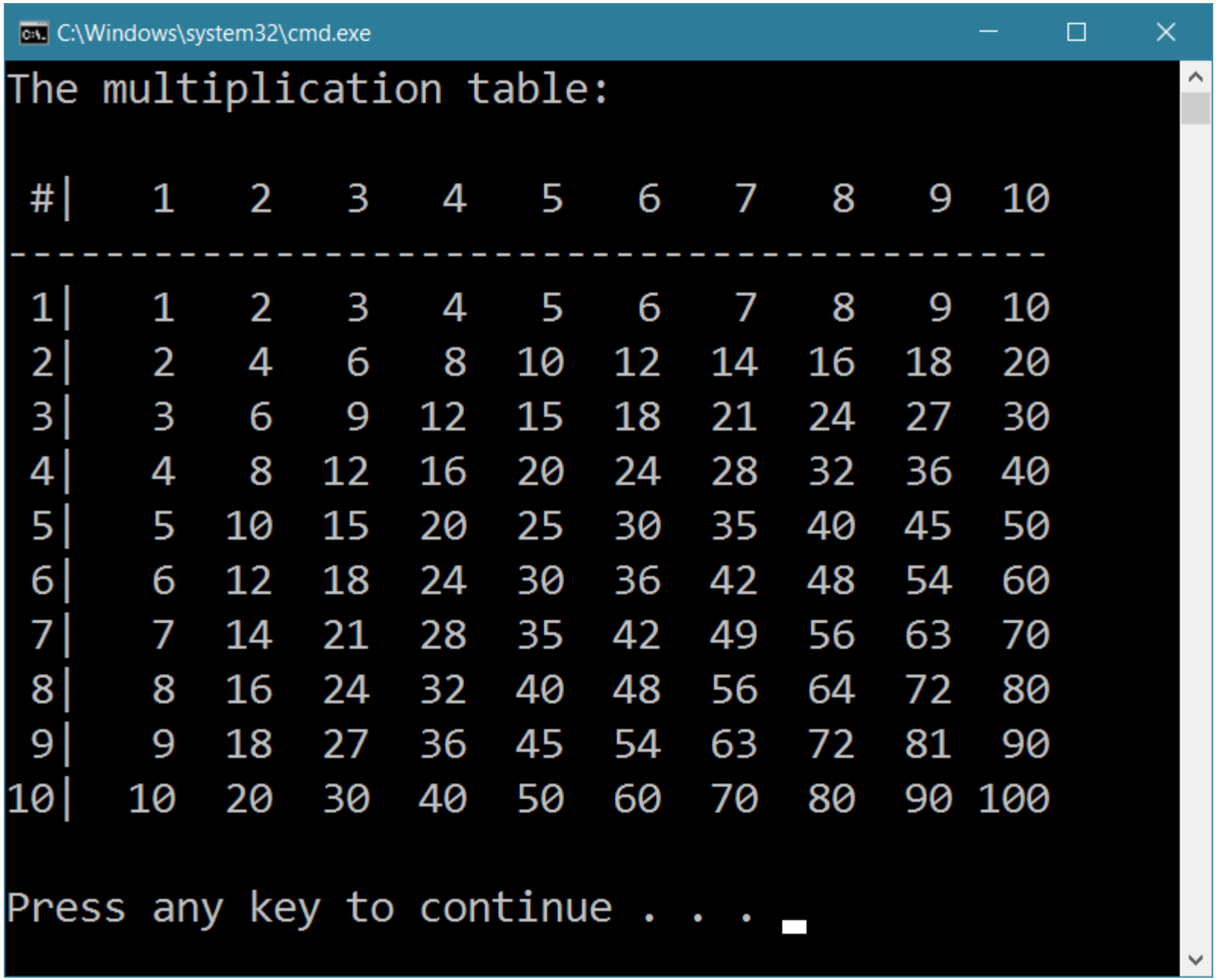
int main()
{
    int x = MyFunction(); // Грешка!
}
```



Пример

Генериране на таблицата за умножение

- В този пример ще подобрим „приложението“, което създадохме преди известно време, генериращо таблица за умножение.
- Ще разпределим кода във функции, като всяка ще отговаря за различна задача.


C:\Windows\system32\cmd.exe
The multiplication table:
#| 1 2 3 4 5 6 7 8 9 10
1| 1 2 3 4 5 6 7 8 9 10
2| 2 4 6 8 10 12 14 16 18 20
3| 3 6 9 12 15 18 21 24 27 30
4| 4 8 12 16 20 24 28 32 36 40
5| 5 10 15 20 25 30 35 40 45 50
6| 6 12 18 24 30 36 42 48 54 60
7| 7 14 21 28 35 42 49 56 63 70
8| 8 16 24 32 40 48 56 64 72 80
9| 9 18 27 36 45 54 63 72 81 90
10| 10 20 30 40 50 60 70 80 90 100
Press any key to continue . . .

Кодът от предишния път

```
std::cout << "The multiplication table:\n\n";

// Извеждаме числата в заглавния ред
std::cout << " #|";

for (int number = 1; number <= 10; number++)
    std::cout << std::setw(4) << number;

std::cout << std::endl;

// Извеждаме линията под числата
for (int count = 1; count <= 3 + 10 * 4; count++)
    std::cout << "-";

std::cout << std::endl;
```

?

?

?





```
// Извеждаме таблицата
for (int lhs = 1; lhs <= 10; lhs++)
{
    std::cout << std:: setw(2) << lhs << '|';

    for (int rhs = 1; rhs <= 10; rhs++)
        std::cout << std::setw(4) << (lhs * rhs);

    std::cout << std::endl;
}

std::cout << std::endl;
```

Подобряване на цикъла for

```
// Извеждаме линията под числата
for (int count = 1; count <= 3 + 10 * 4; count++)
    std::cout << '-';
```

```
// Извеждаме линията под числата
int LineLength = 3 + 10 * 4;

for (int count = 1; count <= LineLength; count++)
    std::cout << '-';
```

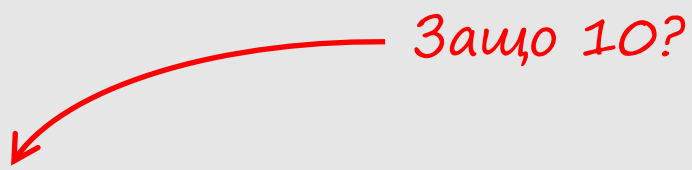
Премахване на „странни“ (магически) числа

```
std::cout << "The multiplication table:\n\n";

// Извеждаме числата в заглавния ред
std::cout << " #|";

for (int number = 1; number <= 10; number++)
    std::cout << std::setw(4) << number;

std::cout << std::endl;
```



```
// Въвеждаме константа за размер на таблицата
const int TABLE_SIZE = 10;

std::cout << "The multiplication table:\n\n";

// Извеждаме числата в заглавния ред
std::cout << " #|";

for (int number = 1; number <= TABLE_SIZE; number++)
    std::cout << std::setw(4) << number;

std::cout << std::endl;
```





```
// Извеждаме линията под числата
int LineLength = 3 + TABLE_SIZE * 4;

for (int count = 1; count <= LineLength; count++)
    std::cout << '-';

std::cout << std::endl;
```





```
// Извеждаме таблицата
for (int lhs = 1; lhs <= TABLE_SIZE; lhs++)
{
    std::cout << std::setw(2) << lhs << '|';

    for (int rhs = 1; rhs <= TABLE_SIZE; rhs++)
        std::cout << std::setw(4) << (lhs * rhs);

    std::cout << std::endl;
}
```

```
const int TABLE_SIZE = 10;

// Въвеждаме константа за размер на клетка в таблицата
const int FIELD_SIZE = 4;

std::cout << "The multiplication table:\n\n";

// Извеждаме числата в заглавния ред

std::cout << " #|";

for (int number = 1; number <= TABLE_SIZE; number++)
    std::cout << std::setw(FIELD_SIZE) << number;

std::cout << std::endl;
```





```
// Извеждаме линията под числата
int LineLength = 3 + TABLE_SIZE * FIELD_SIZE;

for (int count = 1; count <= LineLength; count++)
    std::cout << '-';

std::cout << std::endl;
```





```
// Извеждаме таблицата
for (int lhs = 1; lhs <= TABLE_SIZE; lhs++)
{
    std::cout << std::setw(2) << lhs << '|';

    for (int rhs = 1; rhs <= TABLE_SIZE; rhs++)
        std::cout << std::setw(FIELD_SIZE) << (lhs * rhs);

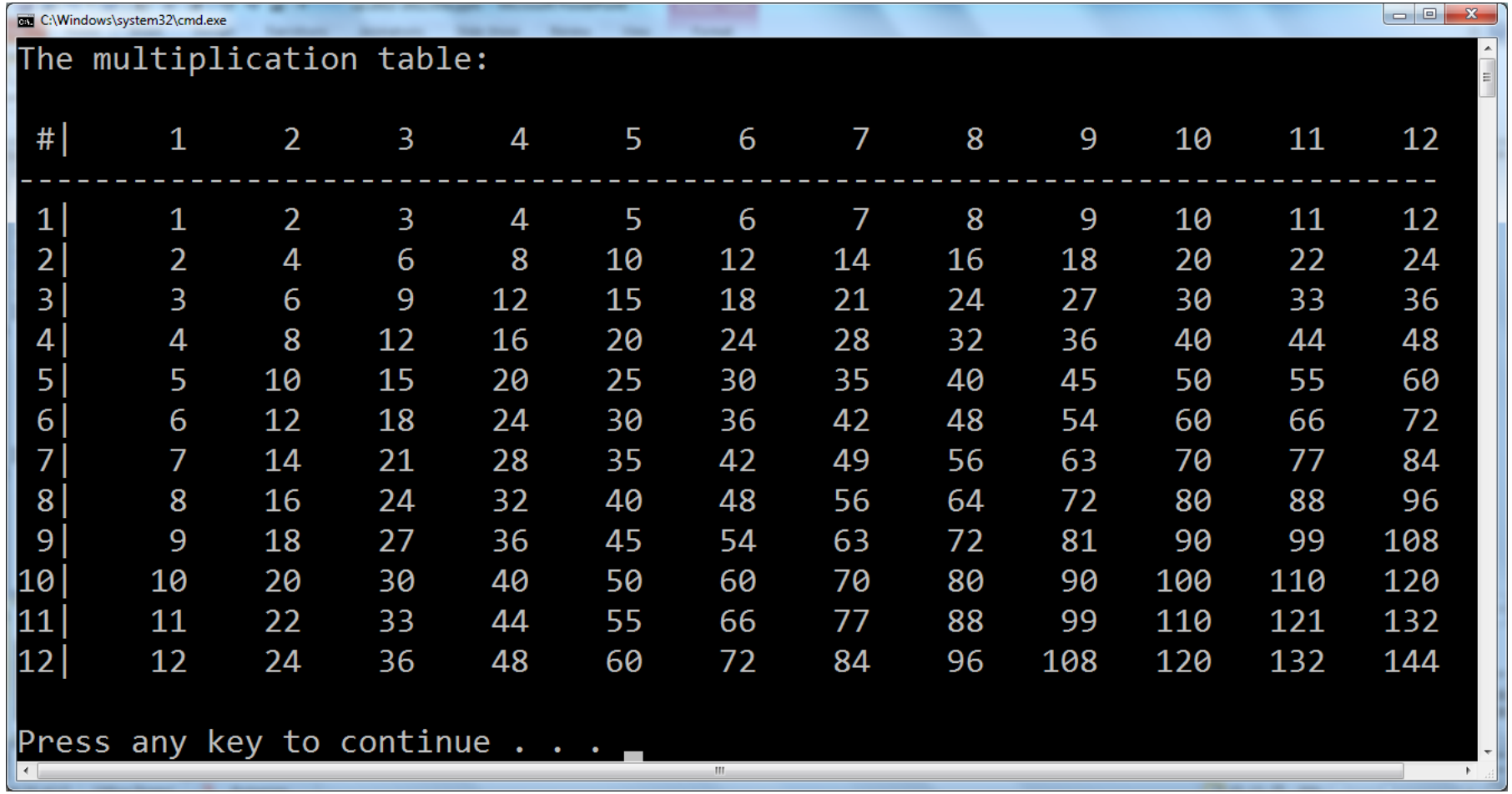
    std::cout << std::endl;
}

std::cout << std::endl;
```

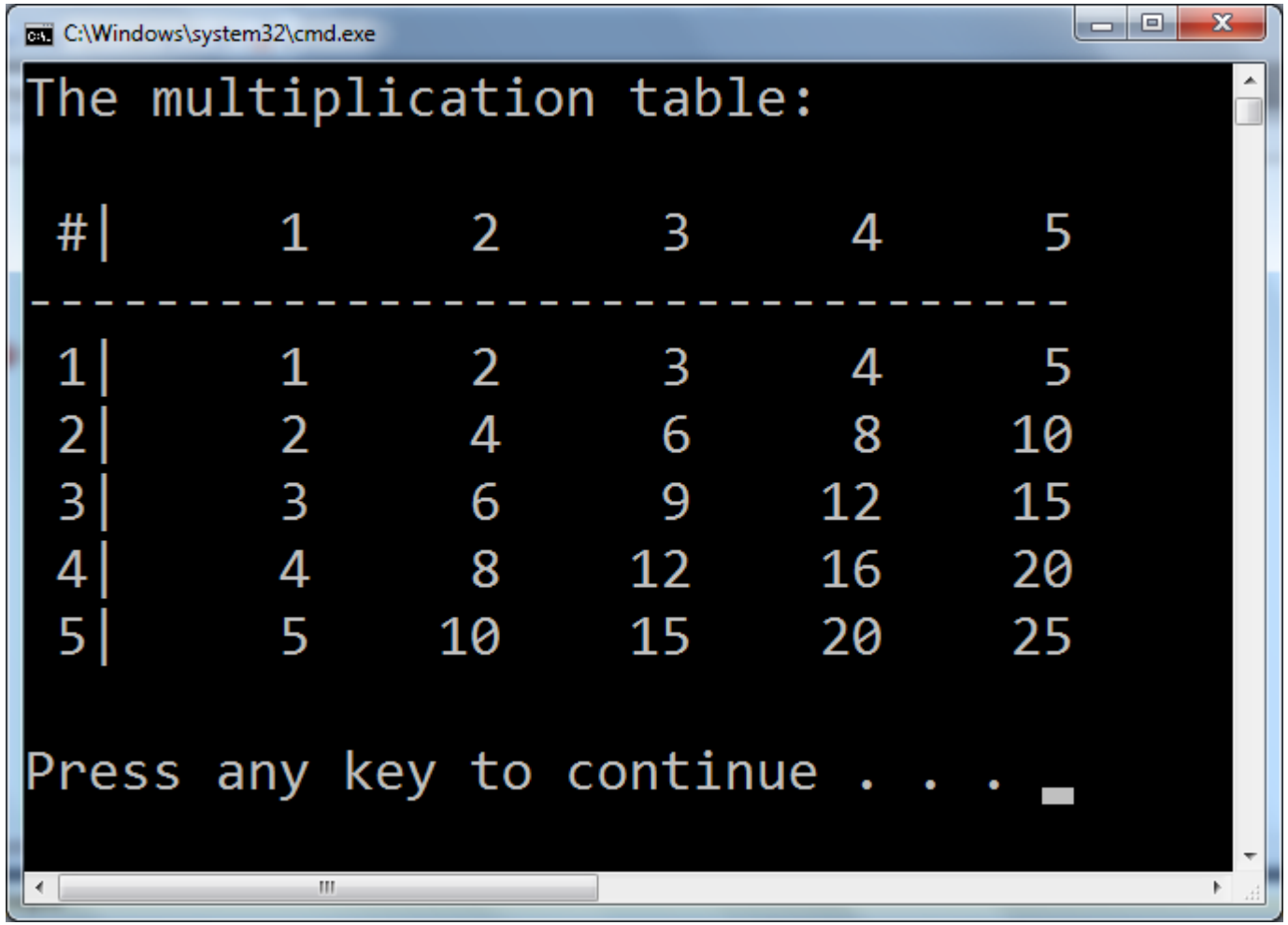
Въпрос

- Защо фиксираме размерът на таблицата? Не е ли по-добре да оставим потребителя да избере такъв?
- Ако искаме да можем да генерираме таблица с произволен размер, ще ни върши ли работа фиксираният размер от 4 позиции за числата в таблицата?

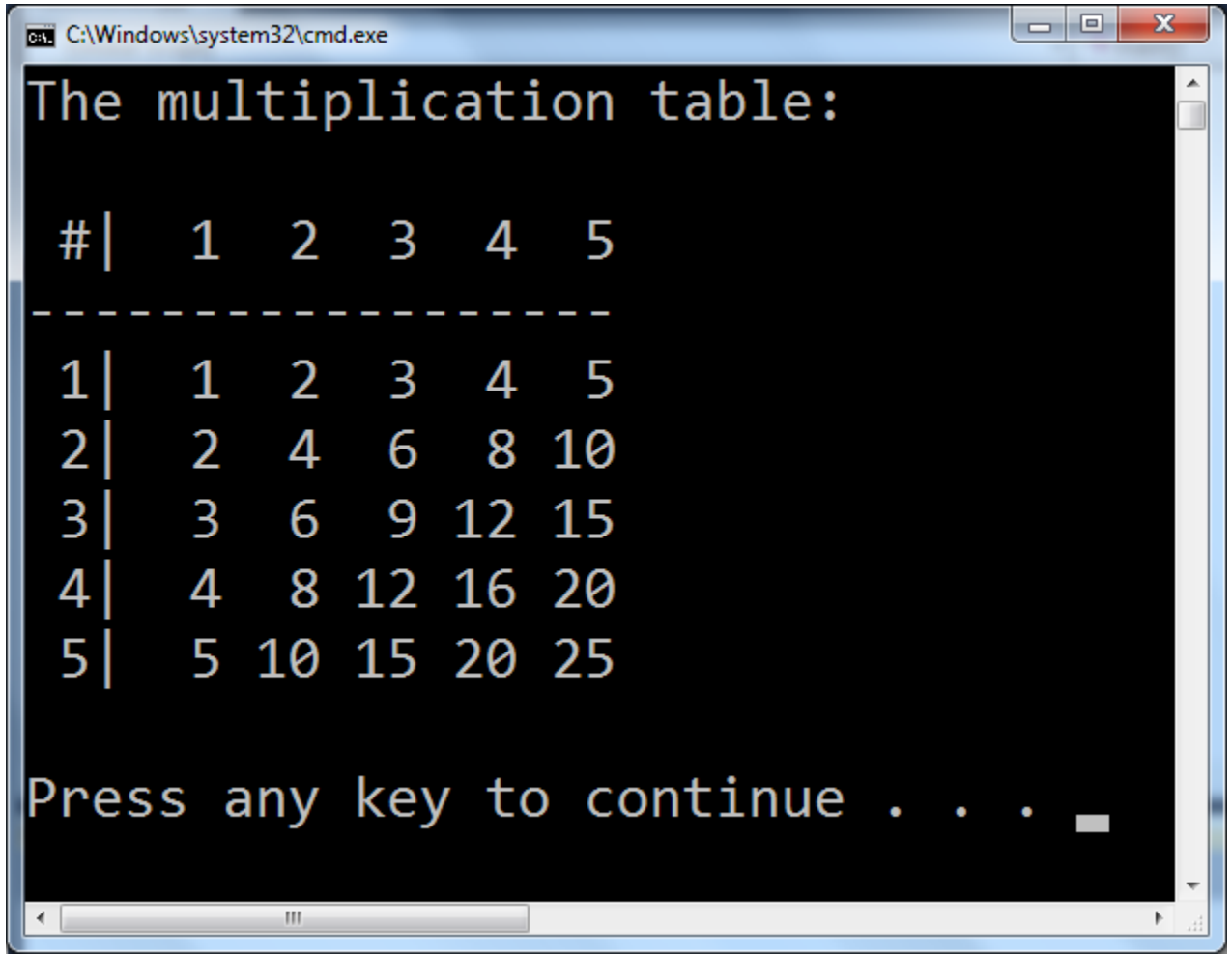
Размер 12x12 и увеличено поле



Размер 5x5 и запазено поле



Размер 5x5 и намалено поле



Контрол на входа

```
int TableSize = 0;

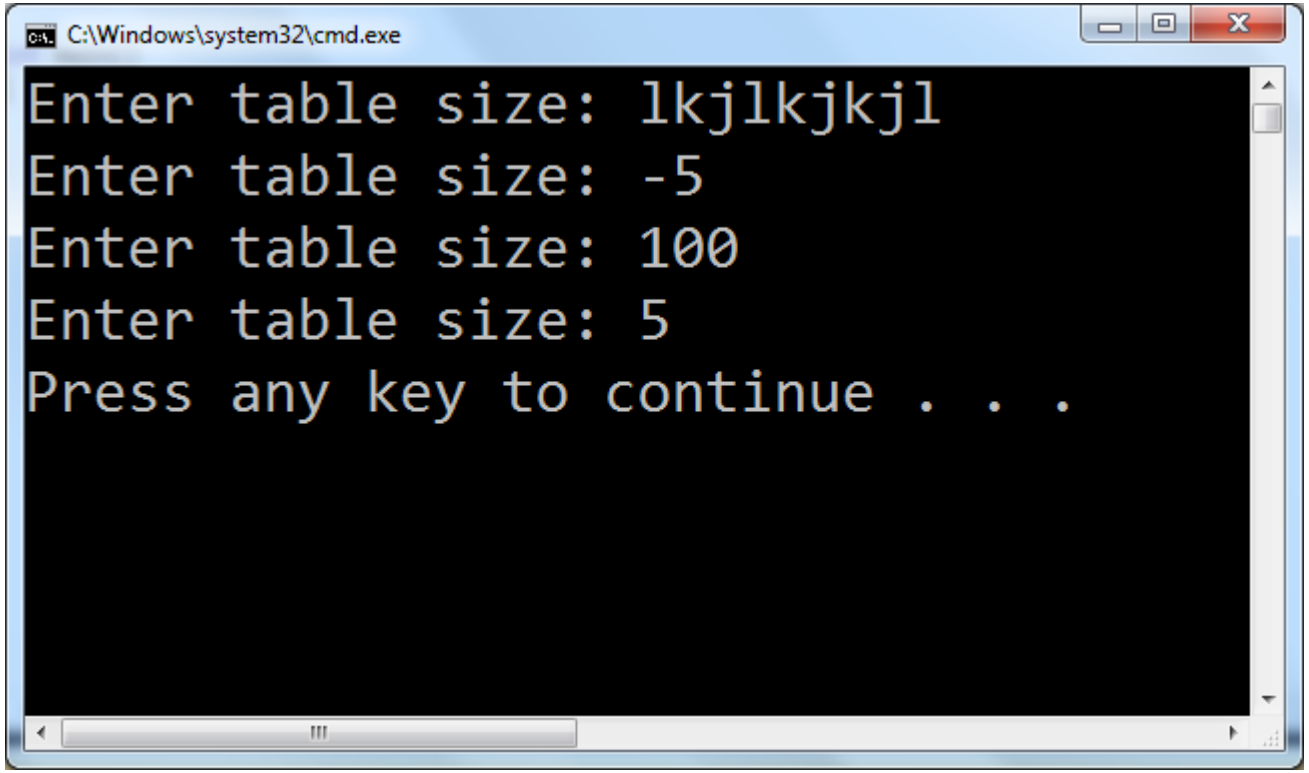
// Възможно е потребителят да въведе невалидна стойност
std::cout << "Enter table size: ";
std::cin >> TableSize;

// ----------------------------------------------
int TableSize = 0;

do
{
    std::cout << "Enter table size: ";
    std::cin >> TableSize;
    if (cin.fail()) {
        cin.clear();
        cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
       continue;
    };

} while (TableSize <= 1 || TableSize >= 12);
```

Какво бихме искали да прави приложението



Ширина на една клетка от таблицата

- Формула:

```
Ширина = Максимален размер на число +
         Отстъп между числата
```

```
int MaxNumberLength =
  (int) log10((double)(TableSize * TableSize)) + 1;

const int FIELD_MARGIN = 2;

int FieldSize = MaxNumberLength + FIELD_MARGIN;
```

Допълнителни Функции

Брой цифри в запис на число

```
int NumberSize(int Number)
{
    int Absolute = (Number >= 0) ?
        Number :
        -Number;

    return (int)log10((double)Absolute) + 1;
}
```

```
// Често една и съща функционалност може
// да се имплементира по различни начини
int NumberSize(int Number)
{
    int Size = 1;
    int Current = (Number >= 0) ? Number : -Number;

    while (Current > 9)
    {
        Current = Current / 10;
        Size++;
    }

    return Size;
}
```

Реализиране на входа

```
// Въвеждане на число в даден интервал
int PromptForInteger(int LowerLimit, int UpperLimit)
{
    int Input = 0;

    do
    {
        std::cout << "Enter an integer between "
                  << LowerLimit << " and "
                  << UpperLimit << ": ";
        std::cin >> Input;

    } while (Input < LowerLimit ||
             Input > UpperLimit);

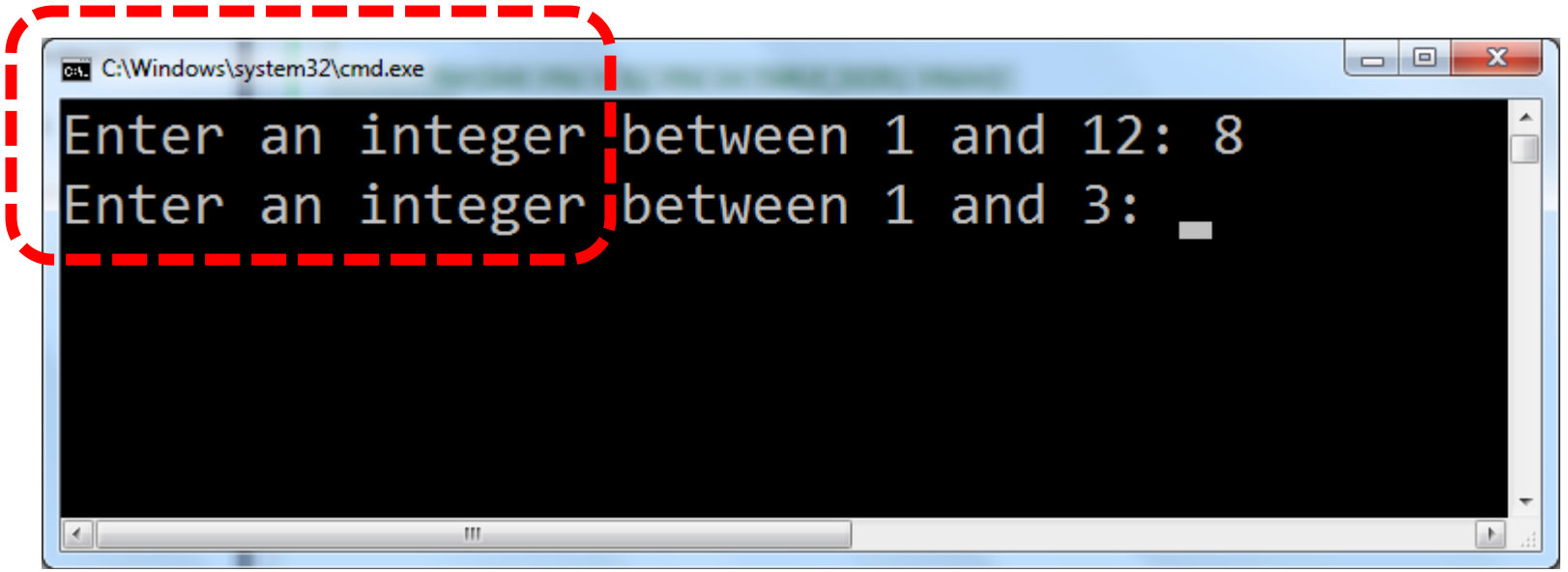
    return Input;
}
```

```
int main()
{
    int TableSize = PromptForInteger(1, 12);
    int FieldMargin = PromptForInteger(1, 3);

    // Това е най-голямото число, което ще се
    // появи в таблицата.
    int MaxNumber = TableSize * TableSize;

    int FieldSize = NumberSize(MaxNumber) +
        FieldMargin;
```

Какво точно се въвежда???



```
int PromptForInteger(const char* Name, int LowerLimit, int UpperLimit)
{
    int Input;

    do
    {
        std::cout << "Enter " << Name
                  << " (between " << LowerLimit
                  << " and " << UpperLimit << "): ";

        std::cin >> Input;

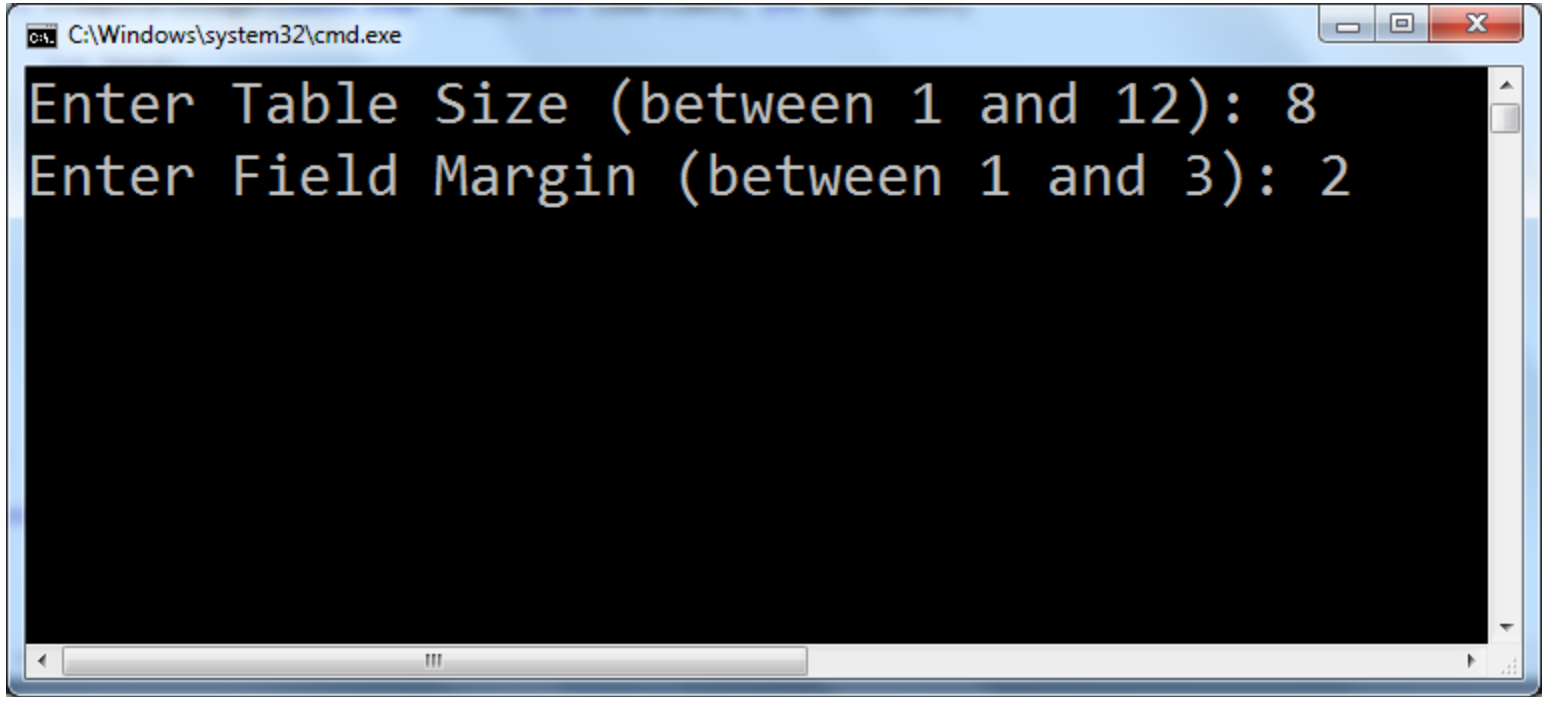
    } while (Input < LowerLimit ||
             Input > UpperLimit);

    return Input;
}
```

```
int main()
{
    int TableSize = PromptForInteger("Table Size", 1, 12);
    int FieldMargin = PromptForInteger("Field Margin", 1, 3);

    // Това е най-голямото число, което ще се появи в таблицата.
    int MaxNumber = TableSize * TableSize;

    int FieldSize = NumberSize(MaxNumber) +
        FieldMargin;
```


C:\Windows\system32\cmd.exe
Enter Table Size (between 1 and 12): 8
Enter Field Margin (between 1 and 3): 2

Други помощни функции

Извеждане на линия

```
void PrintLine(int Length)
{
    for (int i = 0; i < Length; i++)
        std::cout << '-';

    std::cout << std::endl;
}
```

Фрагмент от кода извеждащ заглавния ред в таблицата

```
for (int number = 1; number <= 10; number++)
    std::cout << std::setw(4) << number;
```



```
for (int number = 1; number <= 10; number++)
    std::cout << std::setw(4) << (1 * number);
```

Фрагмент от кода извеждащ един ред в таблицата

```
for (int rhs = 1; rhs <= 10; rhs++)
    std::cout << std::setw(4) << (lhs * rhs);
```

Извеждане на един ред

```
void PrintNumbers(int Start, int End,
                  int Factor, int FieldSize)
{
    for (int i = Start; i <= End; i++)
        std::cout << std::setw(FieldSize) << (i * Factor);

    std::cout << std::endl;
}
```

```
// Извеждаме заглавния ред
std::cout << " #|";
PrintNumbers(1, TableSize, 1, FieldSize);
PrintLine(3 + TableSize * FieldSize);

// Извеждаме таблицата
for (int row = 1; row <= TableSize; row++)
{
    std::cout << std::setw(2) << row << '|';
    PrintNumbers(1, TableSize, row, FieldSize);
}
std::cout << std::endl;
```

```
void PrintTable(int TableSize, int FieldMargin)
{
    int FieldSize = NumberSize(TableSize * TableSize) + FieldMargin;

    std::cout << "The multiplication table:\n\n";

    // Извеждаме заглавния ред
    std::cout << " #|";
    PrintNumbers(1, TableSize, 1, FieldSize);
    PrintLine(3 + TableSize * FieldSize);

    // Извеждаме таблицата
    for (int row = 1; row <= TableSize; row++)
    {
        std::cout << std::setw(2) << row << '|';
        PrintNumbers(1, TableSize, row, FieldSize);
    }

    std::cout << std::endl;
}
```

Окончателен вид на main()

```
int main()
{
    int TableSize = PromptForInteger("Table Size", 1, 12);

    int FieldMargin = PromptForInteger("Field Margin", 1, 3);

    PrintTable(TableSize, FieldMargin);

    return 0;
}
```

- За подготовката на тази презентация са използвани слайдове на доц. Атанас Семерджиев